

NTRE Key Encapsulation Mechanism x86 Implementation Self-Evaluation Report

1. Basic Algorithm Information

1.1 Algorithm Name and Category

The object of evaluation in this report is the NTRE Key Encapsulation Mechanism. NTRE belongs to the category of public-key cryptographic algorithms and provides the core functionalities of key generation, encapsulation, and decapsulation.

Structurally, NTRE follows a two-stage construction. First, an underlying public-key encryption scheme, NTRE.CPAPKE, is constructed. Then, a key encapsulation mechanism, NTRE.CCAKEM, is obtained through a PKE-to-KEM transformation. The underlying NTRE.CPAPKE is responsible for message encryption and recovery, while NTRE.CCAKEM builds on this foundation to provide chosen-ciphertext security for key encapsulation.

1.2 Functional Scope

Table 1. Functional scope of the NTRE KEM implementation

Function	Interface	Description
Key Generation	KeyGen / kem_keygen	Generates a public key and a secret key
Encapsulation	Encaps / kem_enc	Takes a public key as input and outputs a ciphertext and a shared secret
Decapsulation	Decaps / kem_dec	Takes a secret key and a ciphertext as input, and recovers the shared secret or returns a failure result

1.3 Implementation Versions

This submission contains two sets of x86 implementations.

Table 2. NTRE implementation versions

Implementation Version	Directory Scope	Purpose
Reference Implementation	Implementations/ Reference_Implementation/NTRE- {128,256,512}	Focuses on logical correctness, code clarity, and ease of understanding
Performance-Optimized Implementation	Implementations/ Optimized_Implementation/NTRE- {128,256,512}	Optimized for x86-64 and AVX2 environments

This report evaluates the reference implementation and the performance-optimized implementation.

1.4 Parameter Sets and Security Levels

This implementation self-evaluation covers three NTRE instances.

Table 3. NTRE instances covered by this report

Algorithm Instance	Security Level	Implementation Versions
NTRE-128	128-bit	Reference / Optimized
NTRE-256	256-bit	Reference / Optimized
NTRE-512	512-bit	Reference / Optimized

Note: In this report, NTRE-128, NTRE-256, and NTRE-512 refer to the instance names used in the implementation code and test data.

2. Evaluation Basis and Scope

2.1 Evaluation Basis

This report is prepared in accordance with the x86 Architecture Implementation Self-Evaluation Guidelines for Next-Generation Commercial Cryptographic Algorithms. The evaluation covers functional correctness, performance metrics, resource consumption, and communication and storage overhead.

2.2 Evaluation Scope

The evaluation scope includes:

1. The reference implementation of the three NTRE parameter sets;
2. The performance-optimized implementation of the three NTRE parameter sets;
3. Known-answer test vectors;
4. CPU cycle counts for the three core operations: KeyGen, Encaps, and Decaps;
5. Data sizes of public keys, secret keys, ciphertexts, and shared secrets;
6. Static binary size and approximate process-level peak RSS;
7. Engineering reference data against the NTRU+ baseline.

2.3 Test Coverage

The functional tests cover NTRE-128, NTRE-256, and NTRE-512, and include both the reference implementation and the performance-optimized implementation. The performance tests cover the three core operations: KeyGen, Encaps, and Decaps. The communication and storage overhead tests cover the sizes of public keys, secret keys, ciphertexts, and shared secrets.

The corresponding evidence files include KAT files under `Test_Vectors/`, the functional verification log `Benchmark_Results/x86_self_eval_functional.log`, performance CSV/log files under `Benchmark_Results/`, and resource-measurement data recorded in `Benchmark_Results/x86_self_eval_resource.csv`.

3. Algorithm and Implementation Overview

3.1 Algorithmic Background

NTRE is a key encapsulation mechanism consisting of an underlying public-key encryption scheme, NTRE.CPAPKE, and an upper-layer key encapsulation mechanism, NTRE.CCAKEM. NTRE.CPAPKE is designed to provide one-way security under chosen-plaintext attacks, namely OW-CPA security. NTRE.CCAKEM is then constructed through a PKE-to-KEM transformation to obtain a KEM secure against chosen-ciphertext attacks.

The underlying computation is based on a polynomial ring structure and mainly involves polynomial sampling, encoding and decoding, number-theoretic transforms, polynomial multiplication, and modular arithmetic. In NTRE.CPAPKE, encryption mainly computes a ciphertext polynomial, while decryption recovers the message through secret-key polynomial multiplication followed by reduction modulo 2. NTRE.CCAKEM further derives messages, randomness, and shared secrets using hash functions, and checks ciphertext validity during decapsulation.

3.2 Reference Implementation

The reference implementation is written in C99. Its main objective is to preserve clear algorithmic logic, complete interfaces, and ease of reproducibility. This version does not introduce platform-specific hand-written assembly or additional hardware-specific optimizations. It is suitable as a baseline for understanding the algorithmic workflow, checking implementation correctness, and reproducing experimental results.

The reference implementation is compiled with the following configuration:

```
gcc -O2 -std=c99 -Wpedantic -Wall -Wextra  
-Wno-unused-parameter -Wno-unused-result
```

The options `-Wno-unused-parameter` and `-Wno-unused-result` are used for compatibility with the current template and benchmark harness. They do not change the optimization semantics of the algorithm.

3.3 Performance-Optimized Implementation

The performance-optimized implementation aims to reduce the latency of core operations and improve throughput per CPU cycle. It is optimized for x86-64 and AVX2 environments. The optimization methods include stronger compiler optimization options, use of AVX2 instruction-set capabilities, link-time optimization, and implementation-level adjustments for polynomial arithmetic hot paths.

The performance-optimized implementation is compiled with the following configuration:

```
gcc -O3 -march=x86-64 -mavx2 -mtune=native -flto  
-fomit-frame-pointer -std=c99 -Wpedantic -Wall -Wextra  
-Wno-unused-parameter -Wno-unused-result
```

3.4 Main Optimization Targets

The current performance-optimized implementation mainly targets polynomial arithmetic, NTT-related computation, base multiplication, and multiply-add paths. According to the self-evaluation data, the optimized implementation achieves cycle reductions in all three core operations, namely KeyGen, Encaps, and Decaps. The KeyGen path shows the most significant optimization effect.

Further engineering optimization may continue to focus on the following hot paths:

```
poly_basemul_add()
poly_basemul()
poly_msg_mod2_to_bytes()
polynomial-arithmetic paths outside hash/XOF invocation structures
```

The raw benchmark data and build/run logs corresponding to the optimization results are recorded in `Benchmark_Results/ntre_optimized_after.csv` and `Benchmark_Results/ntre_optimized_after.log`.

4. Evaluation Environment

4.1 Hardware Environment

Table 4. Hardware environment

Component	Configuration
CPU Model	Intel(R) Core(TM) Ultra 9 275HX
Clock Frequency	2.70 GHz with Turbo Boost
Instruction Sets	MMX, SSE (1, 2, 3, 4.1, 4.2), SSSE3, EM64T, AES, AVX, AVX2, AVX-VNNI, FMA3, SHA
Core Count	8 performance cores + 16 efficiency cores
Memory	64 GB
Single-Core Binding	<code>taskset -c 0</code>

4.2 Software Environment

Table 5. Software environment

Component	Configuration
Operating System	WSL 2 Ubuntu 26.04
Compiler	GCC 15.2.0
Build Tool	CMake 4.2.3
Counter	x86 <code>rdtsc</code>

Component	Configuration
Cycle Adjustment	<code>countergap</code> deducted
Iterations per Round	10000
Number of Rounds	3

4.3 Test Methodology

Performance tests use the x86 `rdtsc` counter to measure CPU cycle counts, with the counter invocation overhead `countergap` deducted. Each core operation is repeated 10000 times per round, and the final result is averaged over 3 rounds.

This report uses cycles/op as the primary performance metric and additionally reports ops/Mcycle as an equivalent throughput metric. This metric does not depend on dynamic CPU frequency and is suitable for reproducible evaluation in x86 environments with Turbo Boost and scheduling variability. If average wall-clock time or wall-clock ops/s is required in subsequent submissions, the same test programs may be rerun on native Linux with fixed CPU frequency and isolated cores.

The performance-test evidence is indexed by `Benchmark_Results/ntre_optimized_after.csv` and `Benchmark_Results/ntre_optimized_after.log`. The NTRU+ engineering reference data is indexed by `Benchmark_Results/ntruplus_ntre_fair_baseline.csv`.

5. Functional Correctness Test

5.1 Test Method

Functional correctness is tested using known-answer test vectors. The test procedure is as follows:

1. For the three NTRE reference implementations and the three NTRE performance-optimized implementations, execute:

```
make clean all test
```

1. For each instance, run:

```
bin/KAT_KEM
```

1. Compare the generated output byte-by-byte with the standard test vectors in `Test_Vectors/`:

```
cmp output/KAT_KEM_NTRE-*.txt Test_Vectors/KAT_KEM_NTRE-*.txt
```

1. Before timing, the benchmark harness performs a KEM round-trip test, a ciphertext tampering failure test, and a sanity check that the failure output is a 64-byte all-zero key.

The functional verification process is recorded in `Benchmark_Results/x86_self_eval_functional.log`. The corresponding known-answer test vectors are stored in `Test_Vectors/KAT_KEM_NTRE-128.txt`, `Test_Vectors/KAT_KEM_NTRE-256.txt`, and `Test_Vectors/KAT_KEM_NTRE-512.txt`.

5.2 KAT Results

Table 6. KAT results

Instance	Number of KAT Cases	Test Vector SHA-256	Result
NTRE-128	10	65cab746a920cf829732bdb6e2c9f2a9caf198d3fa676a48c85f03f897a87af7	Passed
NTRE-256	10	b46885a10bc6931aad20390b23aba3a630991948242f379cc41e72710145157a	Passed
NTRE-512	10	b3608ede368d63bf002e4819139f748c1e387e1354d98de93420d5db99e24402	Passed

5.3 Correctness Conclusion

The reference implementation and the performance-optimized implementation of NTRE-128, NTRE-256, and NTRE-512 all pass the KAT tests, with outputs matching the test vectors byte-for-byte. Key generation, encapsulation, and decapsulation can all be executed correctly, and the exceptional ciphertext handling path also passes the sanity check.

The raw evidence for this part is `Benchmark_Results/x86_self_eval_functional.log`, together with the KAT files under `Test_Vectors/`.

6. Performance Test

6.1 Test Configuration

The main performance table follows the self-evaluation guideline configuration.

Table 7. Performance compilation configuration

Implementation Version	Compilation Configuration
Reference	O2
Optimized	O3 + AVX2 + LTO

The performance metric is the average CPU cycle count per operation, measured in cycles/op. The tested operations are KeyGen, Encaps, and Decaps.

The raw cycle data for the NTRE reference and optimized implementations are recorded in `Benchmark_Results/ntre_optimized_after.csv`. The corresponding build and benchmark log is recorded in `Benchmark_Results/ntre_optimized_after.log`.

6.2 NTRE CPU Cycle Counts

Table 8. NTRE CPU cycle counts

Instance	Implementation	KeyGen	Encaps	Decaps
NTRE-128	Reference	90555	72201	69989
NTRE-128	Optimized	56609	58438	49684
NTRE-256	Reference	154923	129226	133626
NTRE-256	Optimized	81890	80914	62771
NTRE-512	Reference	376672	351723	360075
NTRE-512	Optimized	137912	171718	140883

6.3 Cycle Reduction

Table 9. Cycle reduction of optimized implementation over reference implementation

Instance	KeyGen Reduction	Encaps Reduction	Decaps Reduction
NTRE-128	37.49%	19.06%	29.01%
NTRE-256	47.14%	37.39%	53.02%
NTRE-512	63.39%	51.18%	60.87%

The results show that the performance-optimized implementation achieves clear cycle reductions over the reference implementation across all three parameter sets. The optimization gain for KeyGen becomes more pronounced as the security level increases. For NTRE-512, the KeyGen cycle count is reduced by 63.39%, and the Decaps cycle count is reduced by 60.87%.

6.4 Equivalent Throughput

The equivalent throughput is computed as:

$$1,000,000 / \text{cycles}$$

The unit is ops/Mcycle. This metric does not depend on dynamic CPU frequency and is suitable for reproducibility under the current test environment.

Table 10. Equivalent throughput

Instance	Implementation	KeyGen ops/Mcycle	Encaps ops/Mcycle	Decaps ops/Mcycle
NTRE-128	Reference	11.04	13.85	14.29
NTRE-128	Optimized	17.67	17.11	20.13
NTRE-256	Reference	6.45	7.74	7.48
NTRE-256	Optimized	12.21	12.36	15.93
NTRE-512	Reference	2.65	2.84	2.78

Instance	Implementation	KeyGen ops/Mcycle	Encaps ops/Mcycle	Decaps ops/Mcycle
NTRE-512	Optimized	7.25	5.82	7.10

6.5 Same-Flags Comparison

To separate the effect of source-level or algorithm-level optimization from compiler-option optimization, this report further provides a comparison between the Reference and Optimized implementations under the same optimized compilation configuration. Both implementations in the following table are compiled with the optimized flags, and the values are still 3-round average cycles/op.

Table 11. Same-flags cycle comparison

Instance	Reference KeyGen	Reference Encaps	Reference Decaps	Optimized KeyGen	Optimized Encaps	Optimized Decaps
NTRE-128	84890	58101	53307	56685	58578	49667
NTRE-256	128030	94749	88925	81883	80511	62494
NTRE-512	257766	222576	210579	137192	171258	140945

The corresponding reductions are as follows.

Table 12. Same-flags cycle reduction

Instance	KeyGen Reduction	Encaps Reduction	Decaps Reduction
NTRE-128	33.23%	-0.82%	6.83%
NTRE-256	36.04%	15.03%	29.72%
NTRE-512	46.78%	23.06%	33.07%

These results show that the current performance-optimized implementation has clear source-level or algorithm-level optimization gains in the KeyGen path. Encaps and Decaps also show source-level optimization gains for NTRE-256 and NTRE-512. For NTRE-128, Encaps under the same flags remains at the same order of magnitude as the Reference version, indicating that for the lower-security parameter set, the observed gain on this path mainly comes from the unified optimized compilation configuration.

The raw data and logs for this section are indexed by `Benchmark_Results/ntre_optimized_after.csv` and `Benchmark_Results/ntre_optimized_after.log`.

7. Engineering Reference Against the NTRU+ Baseline

7.1 Reference Configuration

The NTRU+ baseline in this report is used only as an engineering implementation reference and does not represent the original official NTRU+ release performance. The comparison adopts the following configuration:

1. The NTRU+ source code is migrated from the snapshot under `ntruplus/Reference_Implementation/`;
2. The original `ntruplus/` directory is not modified;
3. The API is unified to NGCC-style interfaces: `kem_get_*`, `kem_keygen`, `kem_enc`, and `kem_dec`;
4. The shared secret length is unified to 64 bytes;
5. The compiler, compilation flags, counter, loop count, and benchmark harness are kept consistent;
6. The parameter pairing is based on approximately corresponding security levels available in the current repository and is used only as an engineering-level cross-reference.

It should be emphasized that the pairing between NTRE and NTRU+ does not constitute a strict same-security comparison. In particular, the implemented security strengths of NTRUplus-864 and NTRUplus-1152 are lower than those of the corresponding NTRE-256 and NTRE-512 instances in this report. Therefore, the differences in running cycles and data sizes at higher security levels must be interpreted together with security strength, parameter scale, and implementation configuration. They should not be used as a direct conclusion on the relative efficiency of the algorithms.

Accordingly, this section is intended to present the engineering behavior of the current NTRE implementation under the same test framework, same API adaptation, same shared secret length, and same benchmark harness. It is not intended to provide a strict performance ranking at identical security levels.

The three-round comparison data are recorded in `Benchmark_Results/ntruplus_ntre_fair_baseline.csv`. The current check confirms that `git diff -- ntruplus` is empty, indicating that the engineering reference test did not modify the original `ntruplus/` directory.

7.2 Reference Cycle Data

Table 13. Engineering reference cycle data

Security Level	Implementation	KeyGen	Encaps	Decaps
128-bit	NTRUplus-768 baseline	78119	29770	30301
128-bit	NTRE-128 reference	85109	57975	53081
128-bit	NTRE-128 optimized	56685	58578	49667
179-bit	NTRUplus-864 baseline	141262	50221	51588
256-bit	NTRE-256 reference	127106	94215	88396
256-bit	NTRE-256 optimized	81883	80511	62494
248-bit	NTRUplus-1152 baseline	152740	57063	59916

Security Level	Implementation	KeyGen	Encaps	Decaps
512-bit	NTRE-512 reference	256011	222086	209209
512-bit	NTRE-512 optimized	137192	171258	140945

7.3 NTRE / NTRU+ Cycle Ratios

A ratio below 1 means that NTRE has a lower cycle count under this test configuration, while a ratio above 1 means that NTRE has a higher cycle count under this test configuration.

Table 14. NTRE / NTRU+ cycle ratios

Pairing	Implementation	KeyGen	Encaps	Decaps
NTRE-128 vs NTRUplus-768	NTRE reference / NTRU+	1.09x	1.95x	1.75x
NTRE-128 vs NTRUplus-768	NTRE optimized / NTRU+	0.73x	1.97x	1.64x
NTRE-256 vs NTRUplus-864	NTRE reference / NTRU+	0.90x	1.88x	1.71x
NTRE-256 vs NTRUplus-864	NTRE optimized / NTRU+	0.58x	1.60x	1.21x
NTRE-512 vs NTRUplus-1152	NTRE reference / NTRU+	1.68x	3.89x	3.49x
NTRE-512 vs NTRUplus-1152	NTRE optimized / NTRU+	0.90x	3.00x	2.35x

7.4 Engineering Reference Analysis

Under the current engineering test configuration, the NTRE performance-optimized implementation shows strong performance in KeyGen. The KeyGen cycle count of NTRE-128 optimized is 0.73 times that of the NTRUplus-768 baseline. The KeyGen cycle count of NTRE-256 optimized is 0.58 times that of the NTRUplus-864 baseline. The KeyGen cycle count of NTRE-512 optimized is 0.90 times that of the NTRUplus-1152 baseline. These results show that the current NTRE performance-optimized implementation has achieved competitive engineering behavior in the key generation path.

For Encaps and Decaps, the cycle differences in the table should be interpreted together with parameter security strength and implementation configuration. The security strengths of NTRUplus-864 and NTRUplus-1152 are lower than those of the corresponding NTRE-256 and NTRE-512 instances. Therefore, their parameter scales, running cycles, and data sizes naturally have lower overhead. In other words, this set of data is more suitable as an engineering reference across different security strengths and parameter scales, and should not be interpreted as a strict comparison under identical security levels.

From the perspective of implementation self-evaluation, this reference data demonstrates two points. First, the KeyGen path of the NTRE optimized implementation already has favorable cycle behavior. Second, under higher-security parameters, NTRE still provides reproducible, executable, and comparable encapsulation and decapsulation performance data, which provides a clear basis for further engineering optimization of polynomial-arithmetic hot paths.

The original evidence for this engineering reference is `Benchmark_Results/ntruplus_ntre_fair_baseline.csv`.

7.5 Data Size Reference

Table 15. Data size reference

Algorithm Instance	pk (bytes)	sk (bytes)	ct (bytes)	ss (bytes)
NTRE-128	972	1976	972	64
NTRE-256	1944	3920	1944	64
NTRE-512	3456	6944	3456	64
NTRUplus-768	1152	2336	1152	64
NTRUplus-864	1296	2624	1296	64
NTRUplus-1152	1728	3488	1728	64

In terms of data size, the public key, secret key, and ciphertext lengths of NTRE-128 are all smaller than those of the NTRUplus-768 baseline. The sizes of NTRE-256 and NTRE-512 are larger than those of the corresponding NTRU+ baseline instances. This result should likewise be understood in light of security strength and parameter scale. Since the implemented security strengths of NTRUplus-864 and NTRUplus-1152 are lower than those of the corresponding NTRE-256 and NTRE-512 instances, their smaller data sizes are reasonable at the parameter level and should not be treated as a strict same-security size comparison.

8. Resource Consumption Test

8.1 Test Method

The static binary size is measured by:

```
size -B
```

which reports the `text`, `data`, and `bss` sections. The peak RSS is measured by running the benchmark binary with:

```
/usr/bin/time -f %M
```

The value `peak_rss_kb` is an approximate process-level measurement, including the benchmark harness, libc, heap allocation, and other process-level overhead. It is therefore more suitable as engineering reproducibility data under the current test environment.

The resource-consumption data are recorded in `Benchmark_Results/x86_self_eval_resource.csv`.

8.2 Static Size and Peak RSS

Table 16. Static size and peak RSS

Category	Implementation	Instance	Compilation Configuration	text	data	bss	total	peak RSS KB
NTRE	Reference	NTRE-128	reference_O2	26828	696	208	27732	1836
NTRE	Reference	NTRE-256	reference_O2	27532	696	208	28436	1836
NTRE	Reference	NTRE-512	reference_O2	28500	696	208	29404	1836
NTRE	Optimized	NTRE-128	optimized_O3_AVX2	63411	704	232	64347	1836
NTRE	Optimized	NTRE-256	optimized_O3_AVX2	58708	704	232	59644	1836
NTRE	Optimized	NTRE-512	optimized_O3_AVX2	56251	704	232	57187	1836
NTRUplus	Baseline	NTRUplus-768	fair_O3_AVX2	66227	704	232	67163	1836
NTRUplus	Baseline	NTRUplus-864	fair_O3_AVX2	87643	704	232	88579	1836
NTRUplus	Baseline	NTRUplus-1152	fair_O3_AVX2	72491	704	232	73427	1704

8.3 Resource Consumption Conclusion

For the reference implementation, the static size increases slightly as the security level increases. The total sizes of NTRE-128, NTRE-256, and NTRE-512 are 27732, 28436, and 29404 bytes, respectively. The performance-optimized implementation has a larger static size than the reference implementation due to stronger optimization and AVX2-related implementation components. The total sizes are 64347, 59644, and 57187 bytes, respectively.

The process-level peak RSS of all three NTRE instances is 1836 KB. This value can be used as engineering reproducibility data under the current test environment and describes the resource usage of the implementations under the same benchmark harness.

The original evidence for this section is `Benchmark_Results/x86_self_eval_resource.csv`.

9. Communication and Storage Overhead

9.1 Data Sizes

Table 17. NTRE communication and storage overhead

Algorithm Instance	pk (bytes)	sk (bytes)	ct (bytes)	ss (bytes)
NTRE-128	972	1976	972	64
NTRE-256	1944	3920	1944	64
NTRE-512	3456	6944	3456	64

9.2 Size Analysis

The shared secret length of all three NTRE instances is 64 bytes. As the security level increases, the public key, secret key, and ciphertext lengths of NTRE increase accordingly. Specifically, for NTRE-128, the public

key and ciphertext are both 972 bytes, and the secret key is 1976 bytes. For NTRE-256, the public key and ciphertext are both 1944 bytes, and the secret key is 3920 bytes. For NTRE-512, the public key and ciphertext are both 3456 bytes, and the secret key is 6944 bytes.

These results show that the data sizes of the NTRE implementation scale clearly with the security level, which facilitates engineering selection and deployment evaluation under different security requirements.

The data-size values are also summarized in the implementation self-evaluation data and are consistent with the corresponding KAT files and benchmark harness outputs.

10. Original Evidence Index

The original evidence files corresponding to this self-evaluation are listed below.

Table 18. Original evidence index

File	Content
Benchmark_Results/ ntre_optimized_after.csv	Raw three-round cycle data for the NTRE reference and optimized implementations
Benchmark_Results/ ntre_optimized_after.log	NTRE benchmark build and run log
Benchmark_Results/ ntruplus_ntre_fair_baseline.csv	Three-round engineering reference data comparing the NTRU+ baseline and NTRE reference implementation
Benchmark_Results/ x86_self_eval_resource.csv	Static size and approximate process-level peak RSS data
Benchmark_Results/ x86_self_eval_functional.log	Functional verification log covering <code>make clean all test</code> , KAT execution, and <code>cmp</code> verification
Test_Vectors/KAT_KEM_NTRE-128.txt	KAT file for NTRE-128
Test_Vectors/KAT_KEM_NTRE-256.txt	KAT file for NTRE-256
Test_Vectors/KAT_KEM_NTRE-512.txt	KAT file for NTRE-512

In addition, the current check confirms that:

```
git diff -- ntruplus
```

returns an empty result, indicating that the engineering reference test did not modify the original `ntruplus/` directory.

11. Self-Evaluation Conclusion

This x86 implementation self-evaluation covers three NTRE instances, namely NTRE-128, NTRE-256, and NTRE-512, as well as two implementation versions: the reference implementation and the performance-optimized implementation. The test results show the following.

1. In terms of functional correctness, both the reference implementation and the performance-optimized implementation of all three NTRE instances pass the KAT tests. The outputs match the test vectors byte-for-byte, and the core KEM round-trip procedure is correct.
2. In terms of performance, the performance-optimized implementation achieves clear cycle reductions over the reference implementation for all three core operations: KeyGen, Encaps, and Decaps. The optimization effect is most significant in the KeyGen path. The KeyGen cycle counts of NTRE-128, NTRE-256, and NTRE-512 are reduced by 37.49%, 47.14%, and 63.39%, respectively.
3. In terms of resource consumption, the performance-optimized implementation has a larger static code size than the reference implementation due to stronger compiler optimization and AVX2-related implementation components. The process-level peak RSS of the three instances remains consistent under the current test environment.
4. In terms of communication and storage overhead, the shared secret length of all three NTRE instances is 64 bytes. The public key, secret key, and ciphertext lengths increase with the security level, showing a clear parameter-scaling relationship.
5. In terms of engineering reference data, the NTRE performance-optimized implementation shows strong competitiveness in KeyGen. The Encaps and Decaps reference data against the NTRU+ baseline should be interpreted together with security strength, parameter scale, and API adaptation. In particular, the implemented security strengths of NTRUplus-864 and NTRUplus-1152 are lower than those of the corresponding NTRE-256 and NTRE-512 instances. Therefore, the related cycle-count and size differences do not constitute a strict conclusion on algorithmic efficiency under identical security levels.

In summary, the current NTRE x86 reference implementation is clear, reproducible, and suitable for implementation review. The performance-optimized implementation achieves explicit optimization effects in KeyGen and selected polynomial-arithmetic paths. Overall, the test results show that NTRE correctly performs key generation, encapsulation, and decapsulation on the x86 architecture, and that the performance-optimized implementation achieves stable reductions in CPU cycle counts.